Build a Video Streaming Platform with HLS, Custom Controls, and CDN Integration

Back

Mandatory

Domain

Frontend Development

Skills

Advanced Accessibility (WCAG) | Content Delivery Network Management | DOM Manipulation (Drag and Drop) | Frontend Development | Monitoring | State Management | Video Playback Implementation

Difficulty

Hard

Tools

Cloudflare | Cloudinary | CSS | Docker | HLS.js | HTML | Javascript | Nginx | FFmpeg

Pending Submission

Please submit your work when ready.

TIME REMAINING

3d

Deadline: **18 Apr 2026, 04:59 pm**

Overview | Instructions | Resources | Submit

Objective

Report Issue

Build a high-performance video streaming application using HLS.js. You will learn to transcode video with FFmpeg, implement a custom

Description

1. Introduction: The World of Modern Video Streaming

Welcome to a production-grade frontend engineering challenge. Video content constitutes over 80% of all internet traffic, and the technologies that power seamless, high-quality streaming are sophisticated and highly sought after. Companies like Netflix, YouTube, and Twitch have built empires on their ability to deliver video reliably and efficiently to a global audience. This task immerses you in the core components of that ecosystem: video transcoding, adaptive streaming protocols, custom player development, and content delivery networks (CDNs).

In this project, you will build a complete video delivery pipeline, from preparing the source media to creating a user-facing player with custom controls. You're not just building a simple web page with a `<video>` tag; you are engineering a content delivery system. This involves understanding the entire lifecycle of a video asset, a skill that distinguishes senior frontend and media engineers.

Why This Matters

1. **Adaptive Bitrate Streaming (ABS)**: This is the cornerstone of modern video delivery. It allows the player to dynamically switch between different quality levels (bitrates) based on the user's network conditions. The result is a smooth viewing experience with minimal buffering, whether the user is on a high-speed fiber connection or a spotty mobile network.
2. **Custom Player Controls**: While off-the-shelf players are convenient, most major platforms build their own to control the user experience, integrate analytics, display custom overlays, and optimize performance. Building your own controls forces you to master the HTML5 Media API and manage complex UI state.
3. **Content Delivery Networks (CDNs)**: A CDN is a geographically distributed network of proxy servers. Serving video segments from a CDN minimizes latency by caching content closer to the user, drastically improving load times and reducing the load on the origin server. Understanding how to integrate with a CDN is a fundamental skill for building scalable web applications.

2. Core Technologies Explained

A. FFmpeg: The Video Transcoder

1. **What It Is**: FFmpeg is a free and open-source command-line tool for handling video, audio, and other multimedia files and streams. It

2. **Its Role in This Task**: Your primary use of FFmpeg will be for **transcoding**. You will take a single high-resolution source video file and convert it into multiple versions, each with a different resolution and bitrate (e.g., 1080p, 720p, 480p). This process is the first step in preparing content for adaptive bitrate streaming.
3. **Why It's Important**: Video must be in the correct format and structure for HLS. FFmpeg creates the necessary video segments (`.ts` files) and the manifest file (`.m3u8`) that lists them.

B. HTTP Live Streaming (HLS)

1. **What It Is**: HLS is an adaptive bitrate streaming protocol developed by Apple. It works by breaking the overall video stream into a sequence of small HTTP-based file downloads, or segments. An index file, or **manifest** (with an `.m3u8` extension), describes the available streams, their different bitrates, and the location of each segment.
2. **How It Works**: The video player starts by fetching the master manifest. This file points to other manifests, one for each quality level. The player then chooses a quality level based on the current network bandwidth, downloads its manifest, and begins downloading and playing the video segments listed within it. If bandwidth changes, the player can switch to a different quality level's manifest and start downloading segments from there, usually at the start of the next segment to ensure a smooth transition.

C. hls.js: The HLS Client

1. **What It Is**: `hls.js` is a JavaScript library that implements an HLS client. It relies on Media Source Extensions (MSE) in modern browsers to deliver fragmented MP4 or transport stream segments to a standard HTML5 `<video>` element.
2. **Its Role in This Task**: You will use `hls.js` to handle the complex logic of parsing the HLS manifest, downloading video segments, and feeding them into the browser's video element for playback. Crucially, it also performs the automatic adaptive bitrate switching. Your job is to integrate with its API to build custom UI controls on top of its core functionality.

D. Content Delivery Network (CDN)

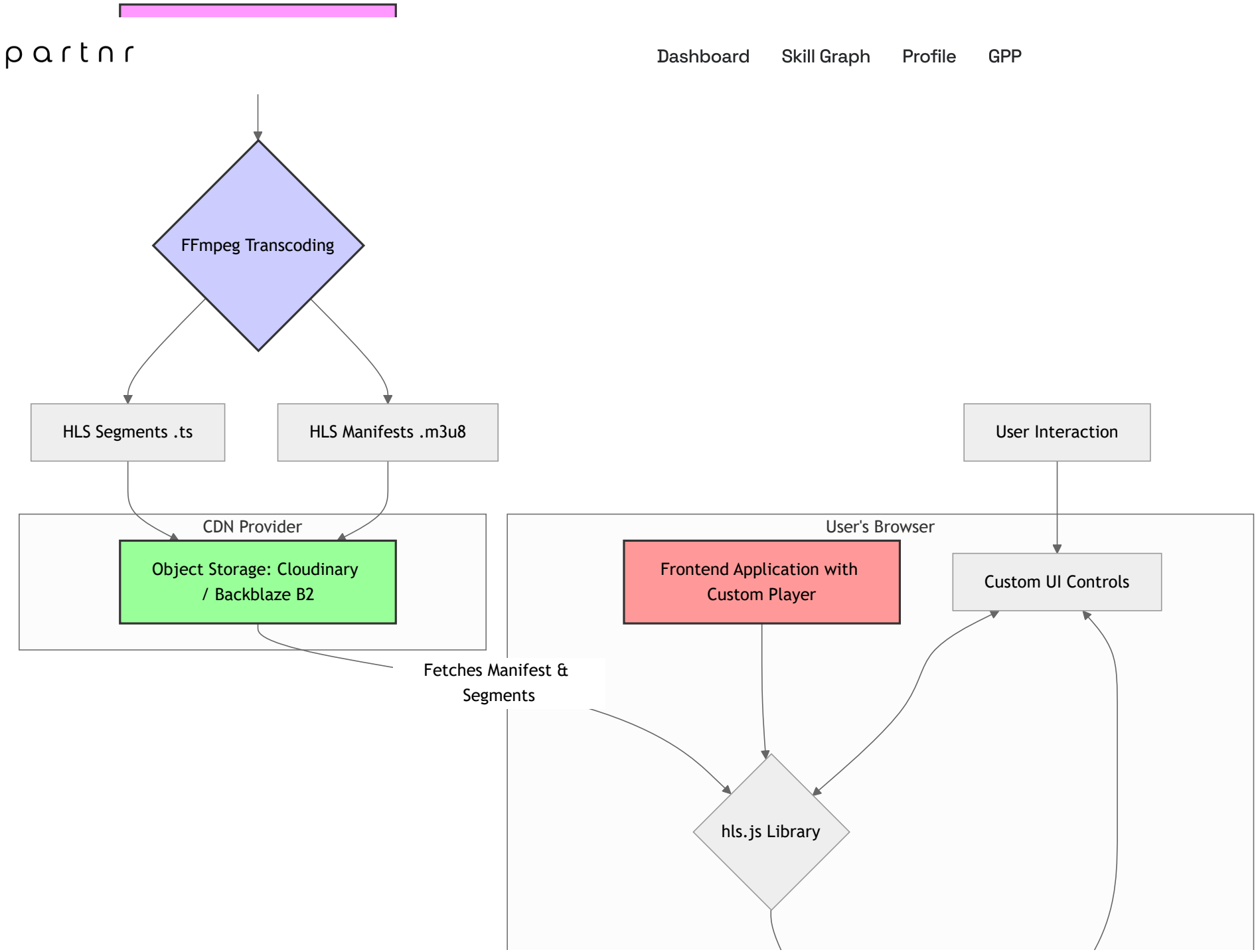
1. **What It Is**: As mentioned, a CDN caches your content (in this case, video segments and manifests) on servers around the world. When a user requests a video, they are served from the server closest to them, known as an edge location.
2. **Its Role in This Task**: You will upload your HLS files to a service like Cloudinary or Backblaze B2, which can be fronted by a CDN like Cloudflare. This ensures that your video segments are delivered quickly and reliably to users anywhere. This step is critical for simulating a real-world production environment.

3. Sustem Architecture and Data Flow

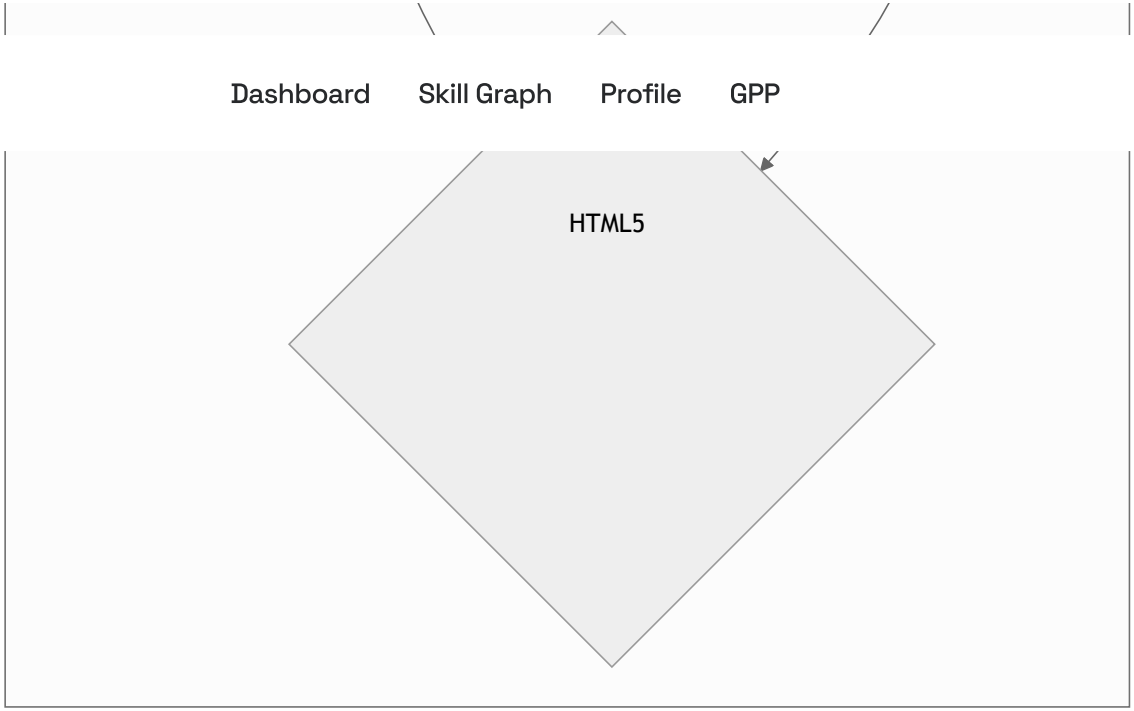
Expand

partnr

Dashboard Skill Graph Profile GPP


partnr
Dashboard
Skill Graph
Profile
GPP
FFmpeg Transcoding
HLS Segments .ts
HLS Manifests .m3u8
User Interaction
CDN Provider
Object Storage: Cloudinary / Backblaze B2
User's Browser
Frontend Application with Custom Player
Custom UI Controls
Fetches Manifest & Segments
hls.js Library

partnr



Architectural Flow Explained:

1. **Preparation (Offline)**: You start with a source video file on your local machine. You use FFmpeg to transcode this file into multiple sets of `.ts` (Transport Stream) segments and their corresponding `.m3u8` manifest files. A master manifest is also created to list all available quality levels.
2. **Distribution**: You upload all the generated `.ts` segments and `.m3u8` manifests to your chosen CDN provider's object storage.
3. **Client-Side Initialization**: The user loads your frontend application. The application initializes `hls.js` and provides it with the URL to the master manifest file hosted on the CDN.
4. **Playback**: `hls.js` fetches and parses the master manifest. It selects an appropriate quality level and begins fetching the video segments. These segments are then passed to the HTML5 `<video>` element via Media Source Extensions.
5. **User Interaction**: The user interacts with your custom UI controls (play, pause, seek, etc.). Your JavaScript code listens for these interactions and uses both the `hls.js` API and the standard HTML5 Media API to control playback.
6. **Adaptation**: `hls.js` continuously monitors the download speed of the video segments. If it detects a change in network conditions, it seamlessly switches to a more appropriate quality stream to prevent buffering.

Phase 1: Video Transcoding with FFmpeg

Your first task is to prepare the media. Download a sample, royalty-free video in a high resolution (e.g., 1080p). You can find suitable videos on sites like Pexels.

1. **Install FFmpeg**: Follow the official instructions to install FFmpeg on your operating system. Verify the installation by running `ffmpeg -version` in your terminal.
2. **Create Transcoding Scripts**: It's best practice to create a shell script to automate this process. Here is an example set of commands. You must adapt them to your source file and desired output.

```
# Example FFmpeg commands for creating an HLS stream
# Replace input.mp4 with your source video file.

SOURCE="input.mp4"
OUTPUT_DIR="hls_output"
mkdir -p $OUTPUT_DIR

# Transcode to different bitrates/resolutions
# 360p
ffmpeg -i $SOURCE -vf "scale=w=640:h=360:force_original_aspect_ratio=decrease" -c:a aac -ar 48000 -c:v h264 -profile:v main -crf 23 -sc_thre

# 480p
ffmpeg -i $SOURCE -vf "scale=w=842:h=480:force_original_aspect_ratio=decrease" -c:a aac -ar 48000 -c:v h264 -profile:v main -crf 23 -sc_thre

# 720p
ffmpeg -i $SOURCE -vf "scale=w=1280:h=720:force_original_aspect_ratio=decrease" -c:a aac -ar 48000 -c:v h264 -profile:v main -crf 23 -sc_thr

# 1080p
ffmpeg -i $SOURCE -vf "scale=w=1920:h=1080:force_original_aspect_ratio=decrease" -c:a aac -ar 48000 -c:v h264 -profile:v main -crf 23 -sc_th

# Create Master Playlist
echo -e "#EXTM3U\n#EXT-X-VERSION:3" > $OUTPUT_DIR/master.m3u8
echo -e "#EXT-X-STREAM-INF:BANDWIDTH=928000,RESOLUTION=640x360\n360p.m3u8" >> $OUTPUT_DIR/master.m3u8
echo -e "#EXT-X-STREAM-INF:BANDWIDTH=1592000,RESOLUTION=842x480\n480p.m3u8" >> $OUTPUT_DIR/master.m3u8
```

```
echo -e "#EXT-X-STREAM-INF:BANDWIDTH=3004000,RESOLUTION=1280x720\n720p.m3u8" >> $OUTPUT_DIR/master.m3u8
```


3. **Document Your Commands**: Create a file named `transcoding.md`. In this file, paste the exact commands you used and add comments explaining what key parameters do (e.g., `-vf scale`, `-b:v`, `-hls_time`, `-g`). This demonstrates your understanding of the transcoding process.

Phase 2: Content Delivery and Setup

1. **Choose and Configure a CDN**: Sign up for a free tier account on Cloudinary or Backblaze B2.
    1. **Cloudinary**: Create a new folder for your project and upload all the `.ts` and `.m3u8` files from your `hls_output` directory. Ensure the files are publicly accessible. Cloudinary has a built-in CDN.
    2. **Backblaze B2 + Cloudflare**: Create a B2 bucket and make it public. Upload your HLS files. Then, sign up for a free Cloudflare account, add your B2 bucket as the origin, and Cloudflare will act as the caching CDN layer in front of it.
2. **CORS Configuration**: This is a critical step. Your CDN/storage bucket must be configured with a Cross-Origin Resource Sharing (CORS) policy that allows your web application's domain to fetch the manifest and segment files. A permissive policy for development might look like this:

```
[
  {
    "allowedHeaders": ["*"],
    "allowedMethods": ["GET", "HEAD"],
    "allowedOrigins": ["*"],
    "exposeHeaders": [],
    "maxAgeSeconds": 3000
  }
]
```

3. **Get the Manifest URL**: Once uploaded, get the public URL for your `master.m3u8` file. This is the URL you will feed into `hls.js`.

Phase 3: Building the Custom Frontend Player

This is the core of the project. You will build the entire player UI from scratch.

1. **HTML Structure**: Start with a simple HTML file containing a `<video>` element and containers for your controls.

```
    <!-- All your custom buttons and sliders go here -->
    <button data-testid="play-pause-button">Play</button>
    <input type="range" data-testid="progress-bar" min="0" max="100" value="0">
    <!-- etc. -->
  </div>
</div>
```

2. **Initialize hls.js**: Include `hls.js` in your project and write the initialization script.

```
const video = document.getElementById('video-player');
const manifestUrl = 'YOUR_CDN_MASTER_MANIFEST_URL.m3u8';

if (Hls.isSupported()) {
  const hls = new Hls();
  hls.loadSource(manifestUrl);
  hls.attachMedia(video);
  hls.on(Hls.Events.MANIFEST_PARSED, function() {
    video.play();
  });
} else if (video.canPlayType('application/vnd.apple.mpegurl')) {
  video.src = manifestUrl;
  video.addEventListener('loadedmetadata', function() {
    video.play();
  });
}
```

3. **Implement Custom Controls (JavaScript Logic)**:

    1. **Play/Pause**: Listen for clicks on your button. Call `video.play()` or `video.pause()`. Listen to the `play` and `pause` events on the video element to keep your UI in sync (e.g., change the button icon).
    2. **Progress Bar/Scrubber**: Listen to the `timeupdate` event on the video element to update the slider's position: `progressBar.value = (video.currentTime / video.duration) * 100`. Also, listen for `input` events on the slider so the user can seek: `video.currentTime = (progressBar.value / 100) * video.duration`.

3. **Volume Control**: Similar to the progress bar, use an input range slider to control `video.volume` (a value from 0.0 to 1.0).

`hls.currentLevel` to the index of the chosen level. Set `hls.currentLevel = -1` to return to automatic adaptive mode.

5. **Fullscreen**: Use the Fullscreen API. Call `videoContainer.requestFullscreen()` on your main container element.
6. **Playback Speed**: Set the `video.playbackRate` property (e.g., 1.0, 1.5, 2.0).

Phase 4: Advanced Features and Accessibility

1. **Watch Progress Persistence**:
    1. Listen to the `timeupdate` event. Every few seconds (don't do it on every single event to avoid performance issues), save `video.currentTime` to `localStorage`: `localStorage.setItem('video-progress', video.currentTime)`.
    2. When the page loads, check if this value exists in `localStorage`. If it does, set `video.currentTime` to the saved value before playing.
    3. Inside the `timeupdate` listener, check if `(video.currentTime / video.duration) >= 0.95`. If true, set a completion flag: `localStorage.setItem('video-completed', 'true')`.
2. **Keyboard Accessibility**: Add a `keydown` event listener to the document. Implement the required shortcuts:
    1. `Space`: Toggle play/pause.
    2. `ArrowRight` / `ArrowLeft`: Seek forward/backward (e.g., `video.currentTime += 5`).
    3. `ArrowUp` / `ArrowDown`: Increase/decrease volume.
    4. `M`: Toggle mute (`video.muted = !video.muted`).
    5. `F`: Toggle fullscreen.

5. Deployment and Verification

For this project, you will use Docker to create a reproducible environment for evaluation.

1. **Dockerfile**: Create a `Dockerfile` that uses a simple web server like `nginx` to serve your static HTML, CSS, and JS files.

```
FROM nginx:alpine
COPY . /usr/share/nginx/html
```

```
EXPOSE 80
```



```
version: '3.8'
services:
  web:
    build: .
    ports:
      - "8080:80"
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost"]
      interval: 30s
      timeout: 10s
      retries: 3
```

3. **Verification**: Run `docker-compose up`. Open your browser to `http://localhost:8080`. Use Chrome DevTools' Network tab to throttle your connection (e.g., to "Slow 3G") and watch the `hls.js` logs or network requests to see the player switch to a lower-quality stream.

6. FAQ

1. **Q: My video is not playing and I see a CORS error in the console. What's wrong?**
    1. **A:** This is the most common issue. Your object storage provider (Cloudinary or B2) is not sending the correct `Access-Control-Allow-Origin` header. Go back to your provider's settings and ensure you have a CORS policy that allows requests from the domain where you are hosting your app (or `*` for development). See the MDN Docs on CORS.
2. **Q: Why do I need to build custom controls? Can't I just use a pre-built player?**
    1. **A:** The goal of this task is to learn the underlying mechanics of video playback and player interaction. Building custom controls forces you to engage directly with the HTML5 Media API and the `hls.js` API, which is an invaluable skill. This level of control is necessary for building unique, highly-branded, or feature-rich video experiences. See the WAI-ARIA Authoring Practices for Media Players for design patterns.
3. **Q: How does `hls.js` actually decide when to switch quality?**

1. **A:** `hls.js` uses a sophisticated algorithm. It continuously measures the user's download bandwidth by timing how long it takes

the current level, it will switch down to avoid buffering. You can explore the hls.js documentation for more details on its internal logic.

Core Requirements

1. The project must be fully containerized using Docker and Docker Compose. A single `docker-compose up` command must build and start all necessary services, including a web server to serve the frontend application. The application must be accessible on a documented port. All services must include a `healthcheck`.

---

2. A markdown file named `transcoding.md` must be present in the root of the repository.

   Verification:

   1. The file `transcoding.md` must exist.
   2. The file must contain the exact `ffmpeg` commands used to transcode the source video into multiple HLS renditions (360p, 480p, 720p, 1080p).
   3. Each command must be accompanied by a brief explanation of the key flags used (e.g., `-vf`, `-b:v`, `-hls_time`).

---

3. All HLS video segments and manifest files must be served from a Content Delivery Network (CDN). The HLS manifest URL used to initialize `hls.js` must point to a Cloudinary or Cloudflare-proxied domain.

   Verification:

   1. The URL of the `master.m3u8` manifest file loaded by the application must be retrieved (e.g., from an environment variable or config file).
   2. The domain of this URL must be verified to be a recognized CDN provider (e.g., `res.cloudinary.com` or a custom domain managed by Cloudflare).

---

4. The video player UI must be built from scratch without using any third-party player UI libraries. All essential controls must be present

.

Control	`data-testid` Attribute
Play/Pause Button	`play-pause-button`
Progress/Scrubber Bar	`progress-bar`
Volume Control Slider	`volume-slider`
Mute Button	`mute-button`
Manual Quality Selector	`quality-selector`
Playback Speed Selector	`playback-speed-selector`
Fullscreen Toggle Button	`fullscreen-button`

Verification:

1. An automated test will query the DOM for each `data-testid` to ensure all required controls are rendered.

---

5. The player must allow the user to manually override the automatic adaptive bitrate selection. The quality selector UI must be dynamically populated with the available quality levels from the HLS manifest.

Verification:

1. After the player loads, the `quality-selector` element must contain options corresponding to the available video resolutions (e.g., '1080p', '720p', 'Auto').
2. Programmatically selecting a quality level (e.g., '480p') must trigger a call to the `hls.js` API to change the active stream (`hls.currentLevel`).
3. An element with `data-testid="current-bitrate-display"` must be present and display the bitrate of the currently playing video segment.

Verification:

1. An automated test will play the video to a specific timestamp (e.g., 30 seconds).
2. The test will check that `localStorage` contains a key (e.g., `video-progress`) with a value approximately equal to 30.
3. The test will reload the page and verify that the video's `currentTime` is automatically set to the stored value upon initialization.

---

7. The application must mark a video as 'completed' in `localStorage` once the user has watched more than 95% of its total duration.

Verification:

1. An automated test will programmatically set the video's `currentTime` to be 96% of its `duration`.
2. The test will check `localStorage` for a specific key (e.g., `video-completed`) to be set to `true`.

---

8. The video player must be fully controllable via the keyboard, adhering to common media player accessibility patterns.

Required Shortcuts:

1. **Space bar**: Toggles play/pause.
2. **'M' key**: Toggles mute.
3. **Right Arrow key**: Seeks forward by 5 seconds.
4. **Left Arrow key**: Seeks backward by 5 seconds.
5. **'F' key**: Toggles fullscreen mode.

Verification:

1. An automated test will focus the player and dispatch `keydown` events for each key.
2. The test will verify that the corresponding action occurs (e.g., pressing 'Space' pauses a playing video, pressing 'ArrowRight' increases `currentTime`).

partnr